TRABAJO PRÁCTICO · UNIDAD 7

Santino Barone – Franco Ríos

Parte 1 - Modelo documental y migración

Consigna 1: Identificación de tablas y esquema de la colección documental

Tablas y claves intervinientes en PostgreSQL:

- **`pedido`**: Tabla principal de cabecera (Clave Primaria: `id`; Clave Foránea: `cliente_id`).
- **`detalle_pedido`**: Tabla intermedia de líneas (Clave Primaria Compuesta: `pedido_id`, `producto_id`; Claves Foráneas a `pedido` y `producto`).
- **`cliente`**: Tabla de entidad (Clave Primaria: `id`).
- **`producto`**: Tabla de catálogo (Clave Primaria: `id`).

Esquema documental propuesto (`pedidos`):

Se diseñó un modelo denormalizado (anidado/embebido) donde cada documento representa un pedido completo, conservando el identificador de origen de PostgreSQL como `_id`.

Consigna 2: Creación de la base de datos y migración

Se creó la base de datos `food_store` y la colección `pedidos` en MongoDB. Se migró un subconjunto de 5 pedidos representativos extrayendo los datos de PostgreSQL y ejecutando la instrucción `db.pedidos.insertMany()` en la consola `mongosh`. Los documentos migrados preservan los identificadores originales (`_id`), fechas, estados, método de pago, datos esenciales del cliente y el desglose de productos con sus cantidades y precios históricos.

Consigna 3: Reglas de integridad y justificación del modelo embebido

Reglas de integridad conservadas:

1. **Inmutabilidad de precios y datos históricos:** Dentro del arreglo `lineas_pedido` se registraron la descripción y el `precio_unitario` al momento exacto de la transacción. Esto asegura que si el precio de un producto cambia a futuro en la tabla relacional de catálogo, la facturación histórica del pedido en MongoDB permanece inalterada.
2. **Preservación de identificadores de origen y dominios válidos:** Se mantuvieron los identificadores numéricos de la base relacional mapeando `pedido.id` a `_id`. Asimismo, se respetó la integridad de los valores de enumerados del dominio (`estado`: *pendiente, pagado, enviado, cancelado*; `forma_pago`: *efectivo, tarjeta_credito, tarjeta_debito, transferencia*).

Justificación del almacenamiento embebido:

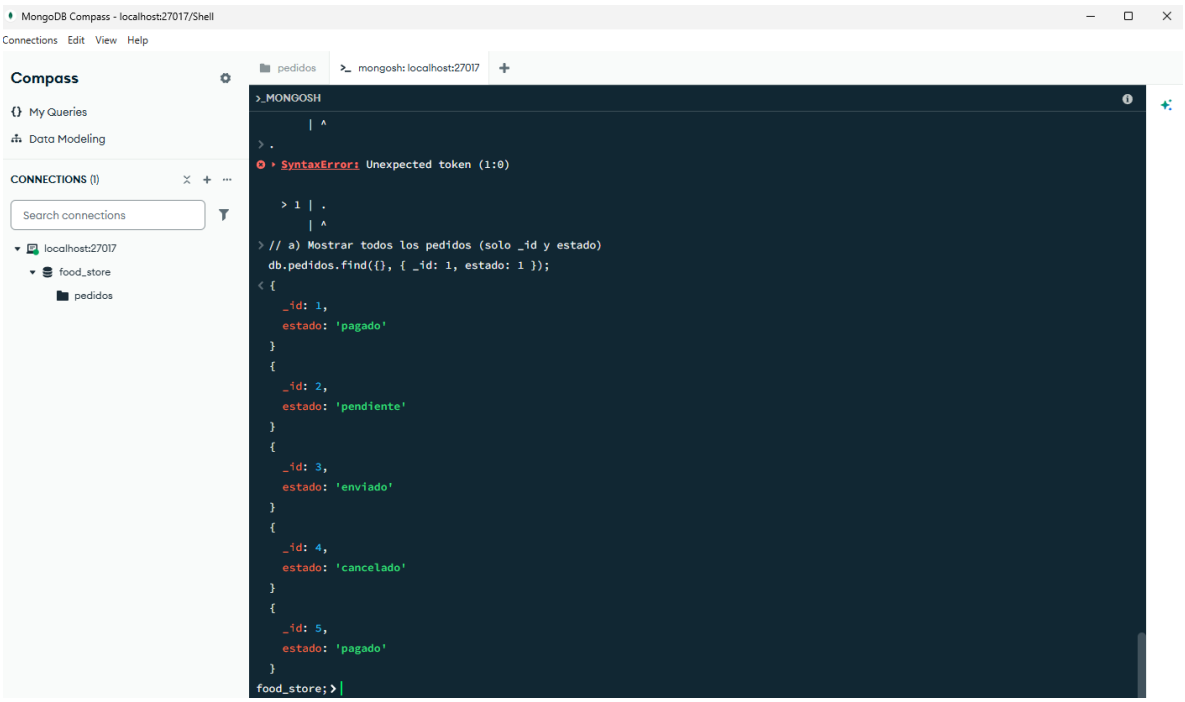
En el sistema *Food Store*, el patrón de acceso preponderante es la lectura completa del pedido junto a sus ítems asociados.

- **Optimización de I/O y lectura:** Embeber las líneas de pedido dentro del documento elimina la necesidad de realizar operaciones de unión complejas (`$lookup` o `JOIN`), reduciendo la latencia al recuperar toda la información en una sola operación de lectura en disco.
- **Atomicidad a nivel de documento:** MongoDB garantiza operaciones atómicas en documentos individuales, lo que asegura que la creación o modificación de un pedido con todas sus líneas se ejecute de forma consistente sin recurrir a transacciones distribuidas entre múltiples colecciones.

- **Parte 2 - Consigna 5: Tabla comparativa de verificación**

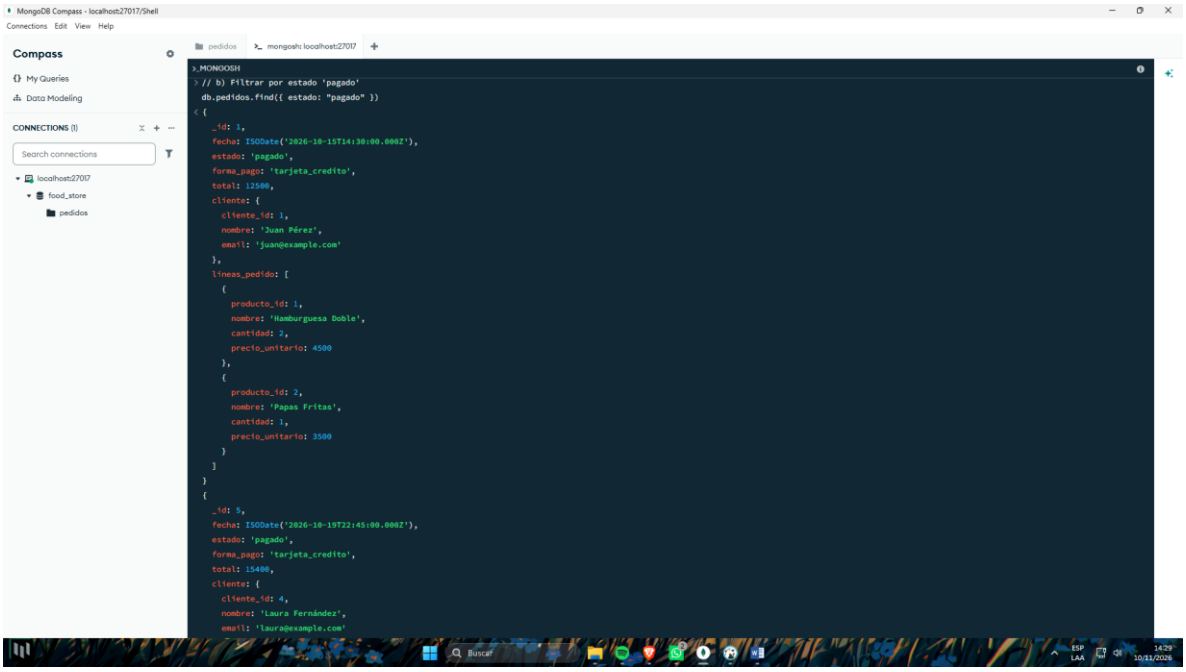
Consulta	Cantidad en PostgreSQL	Cantidad en MongoDB	Coinciden los IDs	Explicación
b) Estado seleccionado (`pagado`)	2	2	Sí (IDs: 1, 5)	Coincidencia exacta de filtrado por atributo de estado.
c) Producto seleccionado (producto_id: 1)	3	3	Sí (IDs: 1, 2, 5)	La consulta sobre el arreglo embebido en MongoDB `lineas_pedido.producto_id` equivale al `JOIN` entre `pedido` y `detalle_pedido` en PostgreSQL.

a) Mostrar todos los pedidos (solo _id y estado)

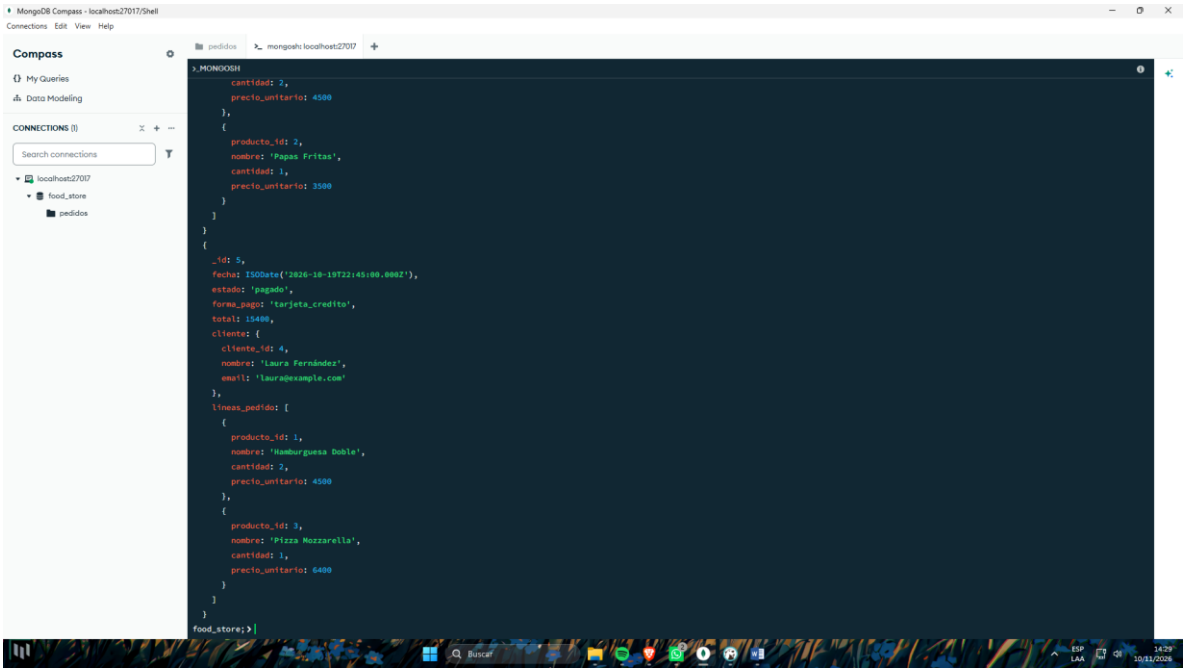


b) Filtrar por estado 'pagado'

1

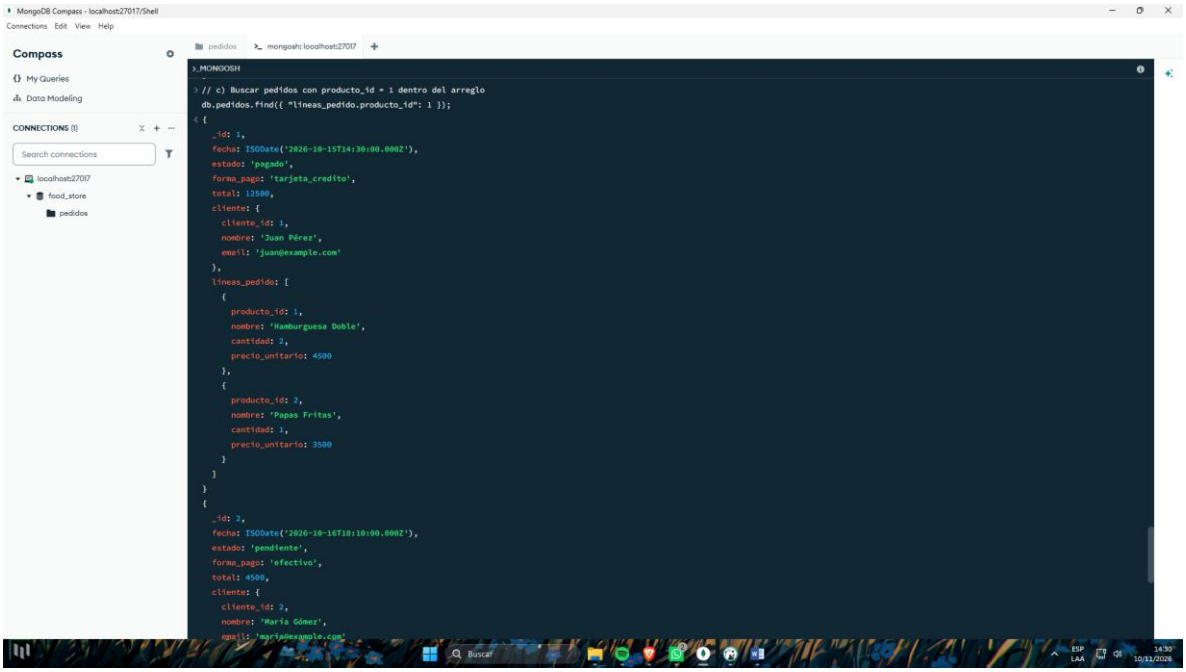


2

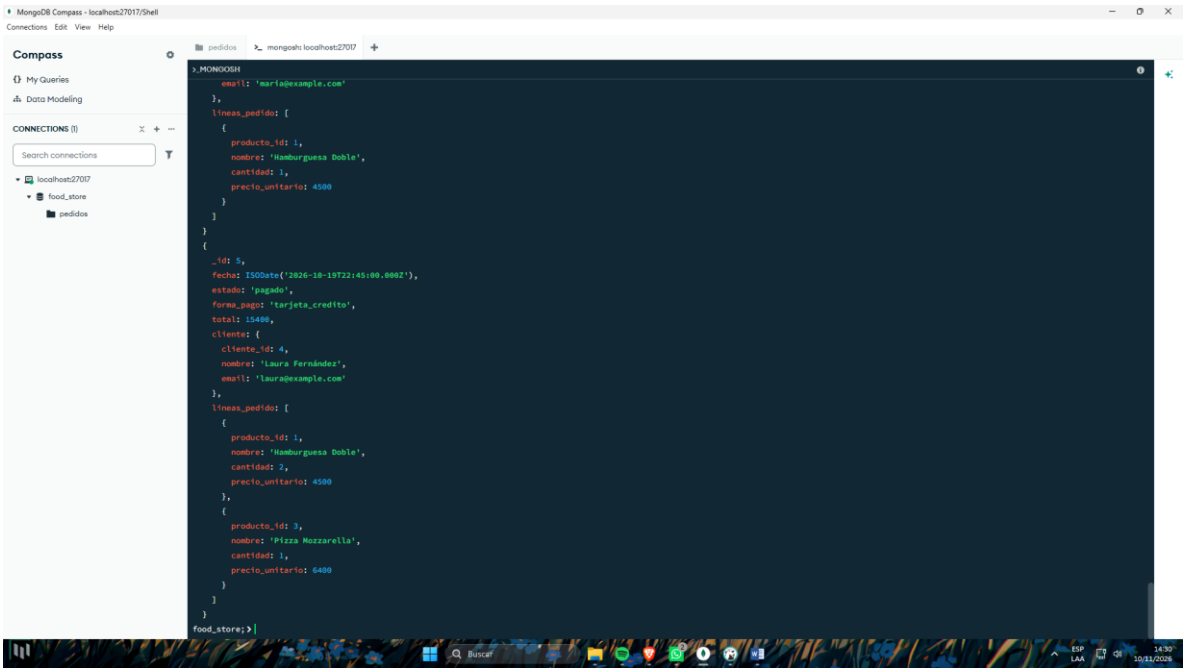


c) Buscar pedidos con producto_id = 1 dentro del arreglo

1

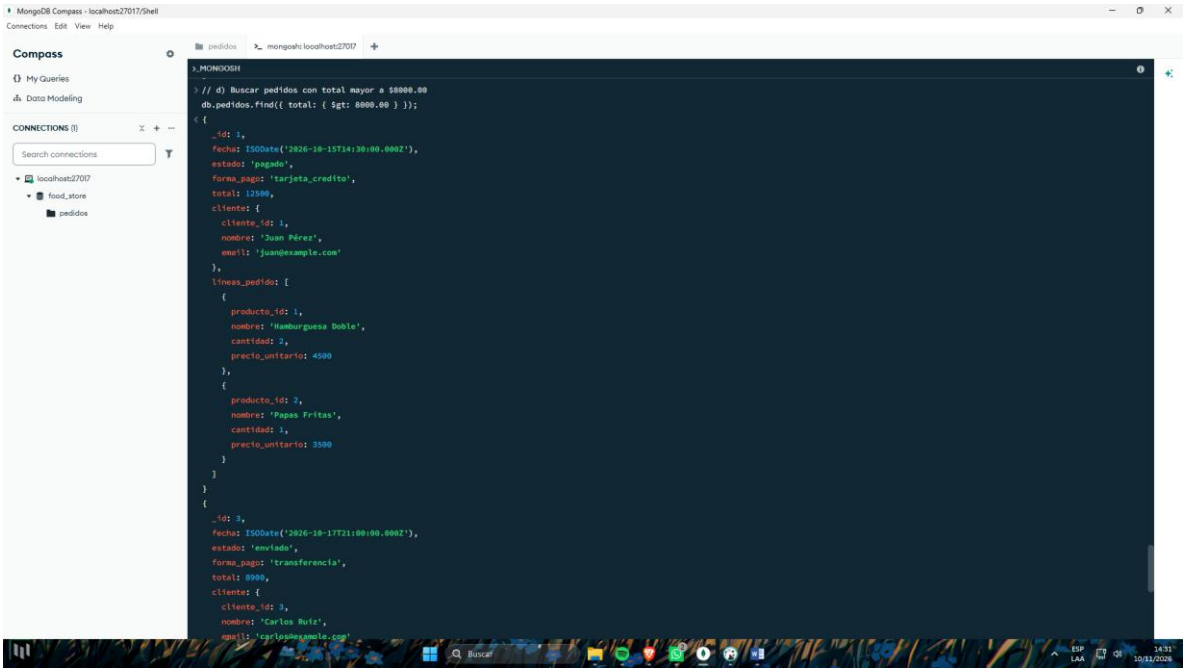


2

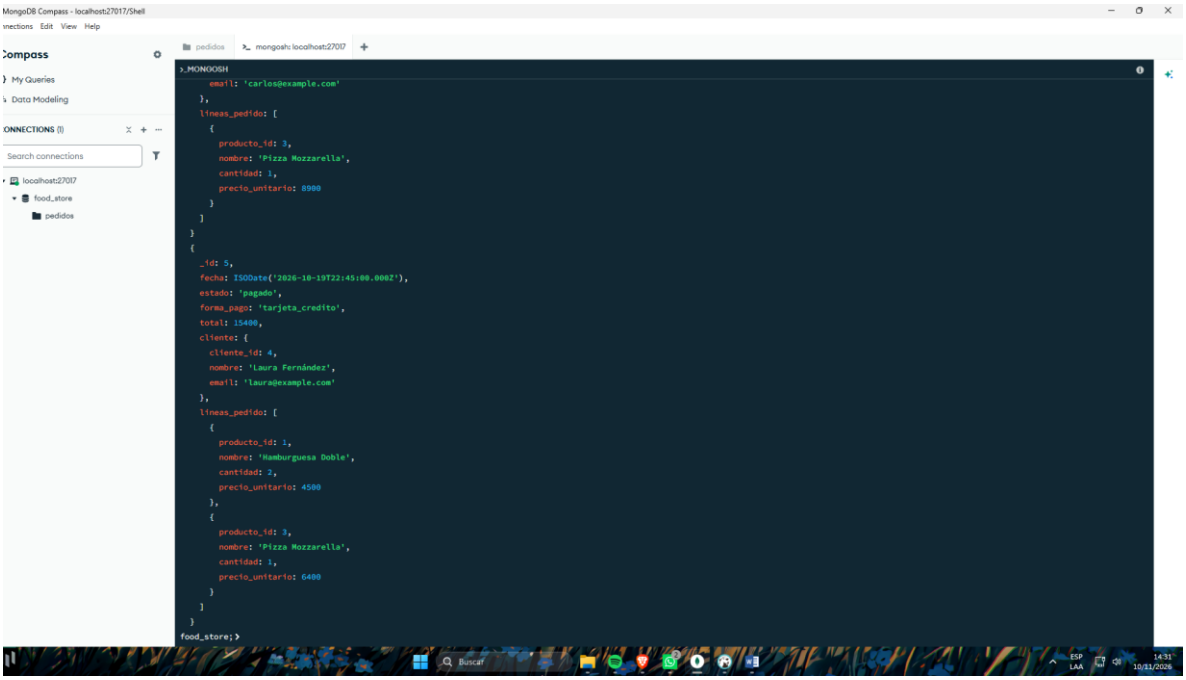


d) Buscar pedidos con total mayor a $8000.00

1


MongoDB Compass - localhost:27017/Shell
Connections Edit View Help
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
food_store
pedidos
pedidos
mongosh: localhost:27017
>_MONGOSH
// d) Buscar pedidos con total mayor a $8000.00
db.pedidos.find({ total: { $gt: 8000.00 } });
{
_id: 1,
fecha: ISODate('2026-10-15T14:30:00.000Z'),
estado: 'pagado',
forma_pago: 'tarjeta_credito',
total: 12500,
cliente: {
cliente_id: 1,
nombre: 'Juan Pérez',
email: 'juan@example.com'
},
lineas_pedido: [
{
producto_id: 1,
nombre: 'Hamburguesa Doble',
cantidad: 2,
precio_unitario: 4500
},
{
producto_id: 2,
nombre: 'Papas Fritas',
cantidad: 1,
precio_unitario: 3500
}
]
}
{
_id: 3,
fecha: ISODate('2026-10-17T21:00:00.000Z'),
estado: 'enviado',
forma_pago: 'transferencia',
total: 8900,
cliente: {
cliente_id: 3,
nombre: 'Carlos Ruiz',
email: 'carlos@example.com'
Buscar
ESP
LAA
14:31
10/11/2026

2


MongoDB Compass - localhost:27017/Shell
Compass
My Queries
Data Modeling
CONNECTIONS (1)
Search connections
localhost:27017
food_store
pedidos
pedidos
mongosh: localhost:27017
>_MONGOSH
email: 'carlos@example.com'
},
lineas_pedido: [
{
producto_id: 3,
nombre: 'Pizza Mozzarella',
cantidad: 1,
precio_unitario: 8900
}
]
}
{
_id: 5,
fecha: ISODate('2026-10-19T22:45:00.000Z'),
estado: 'pagado',
forma_pago: 'tarjeta_credito',
total: 15400,
cliente: {
cliente_id: 4,
nombre: 'Laura Fernández',
email: 'laura@example.com'
},
lineas_pedido: [
{
producto_id: 1,
nombre: 'Hamburguesa Doble',
cantidad: 2,
precio_unitario: 4500
},
{
producto_id: 3,
nombre: 'Pizza Mozzarella',
cantidad: 1,
precio_unitario: 6400
}
]
}
food_store>
Buscar
ESP
LAA
14:31
10/11/2026

Parte 3 - Consignas 6 y 7: Evaluación de IA y Conclusión Técnica

Prompt utilizado para la IA

"Actúa como un arquitecto de bases de datos. Compara PostgreSQL y MongoDB respecto del almacenamiento y la consulta de pedidos para el sistema Food Store. Ten en cuenta que el modelo relacional actual requiere acceso frecuente a los pedidos completos (cabecera y detalle juntos) y es crítico conservar la coherencia de cantidades y precios históricos. Genera un análisis técnico breve indicando ventajas y desventajas."

Contraste de afirmaciones de la IA

1. **Afirmación 1 de la IA:** *"MongoDB es significativamente más rápido para consultar un pedido completo porque evita la sobrecarga computacional de los JOINs relacionales."*
    - **Contraste técnico: Verdadero.** Al verificar la consulta *c* en el laboratorio, en PostgreSQL fue necesario hacer un `JOIN` explícito entre `pedido` y `detalle_pedido` agregando un `DISTINCT`, mientras que en MongoDB la consulta se resolvió buscando directamente en la estructura documental. La documentación oficial de MongoDB sobre modelos embebidos confirma que el anidamiento optimiza las operaciones de lectura al reducir el I/O en disco.
2. **Afirmación 2 de la IA:** *"Al migrar a MongoDB se pierde la capacidad de garantizar la integridad referencial de los datos del cliente y del producto."*
    - **Contraste técnico: Parcialmente Verdadero / Depende de la implementación.** MongoDB no aplica restricciones de clave foránea (*Foreign Keys*) a nivel de motor como PostgreSQL (`REFERENCES cliente(id)`). Sin embargo, esto no implica pérdida de integridad si la lógica de negocio o la validación del esquema (usando *JSON Schema validation* en MongoDB) controlan la inserción de referencias válidas.

Conclusión Técnica

Para el dominio de **Food Store**, la solución óptima no consiste en reemplazar un motor por otro, sino en adoptar una **persistencia políglota**.

Mantener **PostgreSQL** es fundamental para el núcleo transaccional, el catálogo de productos, la gestión de inventario y la facturación global, donde la rigidez del esquema relacional, las claves foráneas y el cumplimiento estricto de ACID evitan inconsistencias como ventas sin stock o clientes inexistentes.

Por otro lado, **MongoDB** resulta ideal para la gestión operativa y de lectura rápida de pedidos activos y del historial de carritos. Embeber las líneas de detalle dentro del documento de pedido optimiza drásticamente la latencia en la aplicación web/móvil al eliminar operaciones de `JOIN`. Además, el almacenamiento del precio histórico dentro del documento garantiza la inmutabilidad de la facturación pasada frente a cambios de tarifas en el catálogo.

En conclusión, se recomienda una arquitectura híbrida: PostgreSQL como fuente primaria de verdad relacional e histórica, y MongoDB como almacén documental para la gestión y consulta de pedidos en tiempo real.